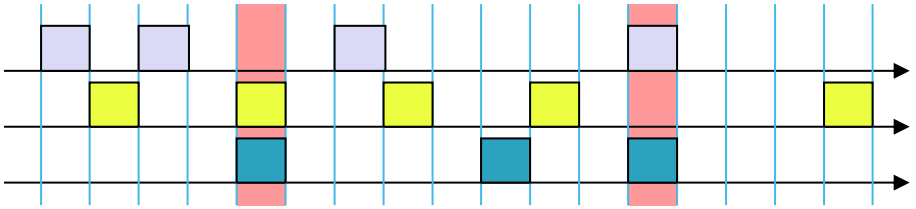
Számítógépes Hálózatok

6. Előadás: Adatkapcsolati réteg
MAC alréteg

Based on slides from **Zoltán Ács ELTE** and D. Choffnes Northeastern U., Philippa Gill from StonyBrook University , Revised Spring 2016 by S. Laki

Réselt ALOHA

- A csatornát azonos időrésekre bontjuk, melyek hossza pont egy keret átviteléhez szükséges idő.
- Átvitel csak az időrések határán lehetséges



- Algoritmus:
  - Amikor egy új A keret küldésre kész:
    - Az A keret kiküldésre kerül a (következő) időrés-határon

A réselt ALOHA vizsgálata

- A sebezhetőségi idő a felére csökken!!!
- Tudjuk, hogy:

$$P_k(t) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$

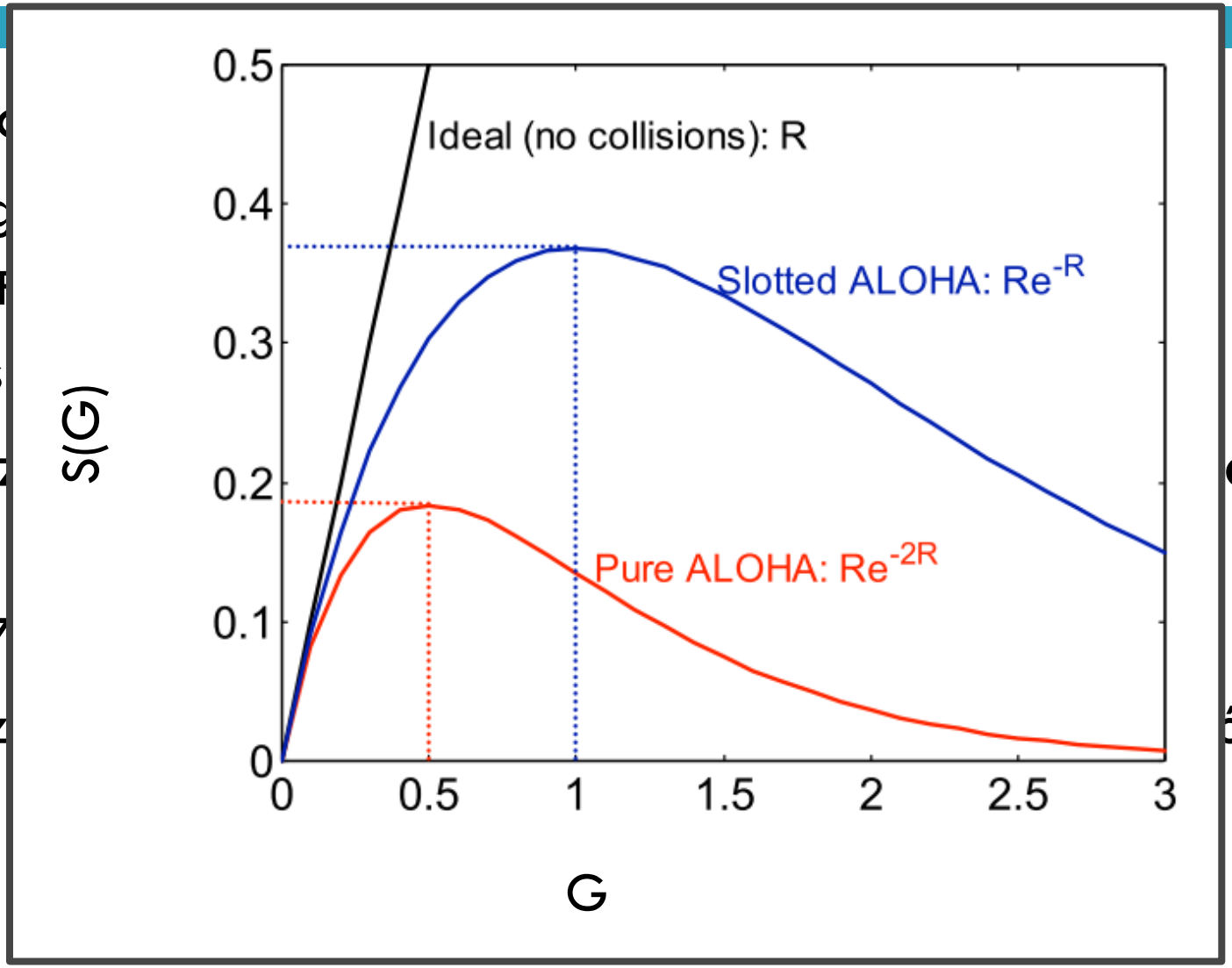
Ez esetben $t = T_f$ és továbbra is $k = 0$, amiből kapjuk, hogy:

$$P_0(T_f) = \frac{(\lambda \cdot T_f)^0 e^{-\lambda T_f}}{0!} = e^{-G}$$

$$\text{because} \quad \lambda = \frac{G}{T_f}. \qquad \text{Thus,} \quad \boxed{S = G \cdot e^{-G}}$$

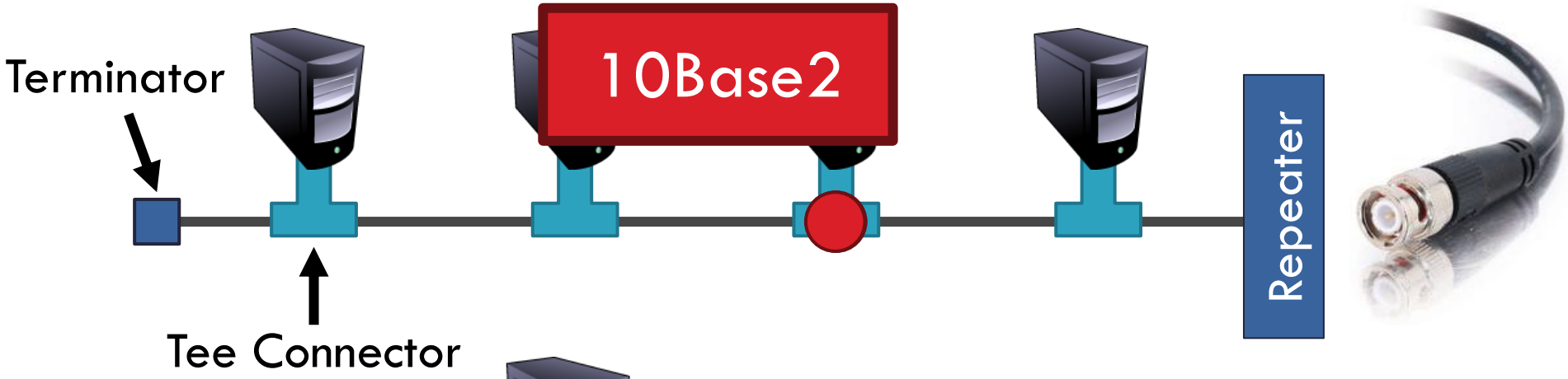
Réselt ALOHA

- Proto
  - Ug
    - F
  - Cs
- Azaz általán nem
  - 37
  - Az órával ke



Adatszóró (Broadcast) Ethernet

- Eredetileg az Ethernet egy adatszóró technológia volt



- 10BaseT és 100BaseT
- T jelzi a csavart érpárt (Twisted Pair)

b

Hubok és repeaterek mind 1. rétegbeli eszközök (csak fizikai)

Vivőjel érzékelés
Carrier Sense Multiple Access (CSMA)

- További feltételezés
  - Minden állomás képes belehallgatni a csatornába és így el tudja dönteni, hogy azt más állomás használja-e átvitelre

1-perzisztens CSMA protokoll

- Vivőjel érzékelés van, azaz minden állomás belehallgathat a csatornába.
- Folytonos időmodellt használ a protokoll

Algoritmus

- Keret leadása előtt belehallgat a csatornába:
  a) Ha foglalt, akkor addig vár, amíg fel nem szabadul. Szabad csatorna esetén azonnal küld. (*perzisztens*)
  b) Ha szabad, akkor küld.
- Ha ütközés történik, akkor az állomás véletlen hosszú ideig vár, majd újrakezdi a keret leadását.

Tulajdonságok

- A terjedési késleltetés nagymértékben befolyásolhatja a teljesítményét.
- Jobb teljesítményt mutat, mint az ALOHA protokollok.

Nem-perzisztens CSMA protokoll

- Vivőjel érzékelés van, azaz minden állomás belehallgathat a csatornába.
- Folytonos időmodellt használ a protokoll
- Mohóság kerülése

Algoritmus

- Keret leadása előtt belehallgat a csatornába:
  a) Ha foglalt, akkor véletlen ideig vár (nem figyeli a forgalmat), majd kezdi előről a küldési algoritmust. (*nem-perzisztens*)
  b) Ha szabad, akkor küld.
- Ha ütközés történik, akkor az állomás véletlen hosszú ideig vár, majd újrakezdi a keret leadását.

Tulajdonságok

- Jobb teljesítményt mutat, mint az 1-perzisztens CSMA protokoll. (*intuitív*)

p-perzisztens CSMA protokoll

- Vivőjel érzékelés van, azaz minden állomás belehallgathat a csatornába.
- Diszkrét időmodellt használ a protokoll

Algoritmus

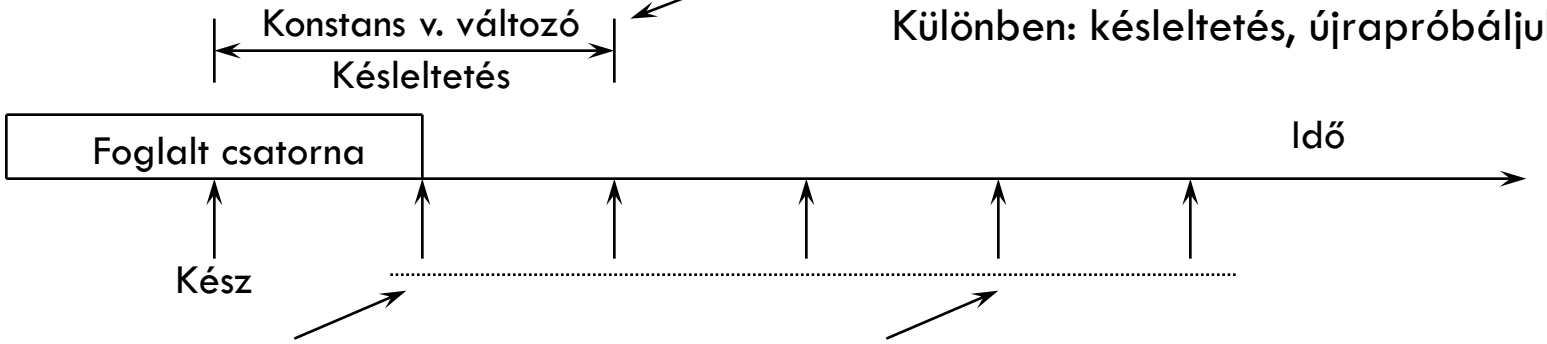
- Adás kész állapotban az állomás belehallgat a csatornába:
  a) Ha foglalt, akkor vár a következő időrésig, majd megismétli az algoritmust.
  b) Ha szabad, akkor *p* valószínűséggel küld, illetve *1-p* valószínűséggel visszalép a szándékától a következő időrésig. Várakozás esetén a következő időrésben megismétli az algoritmust. Ez addig folytatódik, amíg el nem küldi a keretet, vagy amíg egy másik állomás el nem kezd küldeni, mert ilyenkor úgy viselkedik, mintha ütközés történt volna.
- Ha ütközés történik, akkor az állomás véletlen hosszú ideig vár, majd újrakezdi a keret leadását.

CSMA áttekintés

- Nem-perzisztens
- 1-perzisztens
- *p*-perzisztens

CSMA perzisztencia

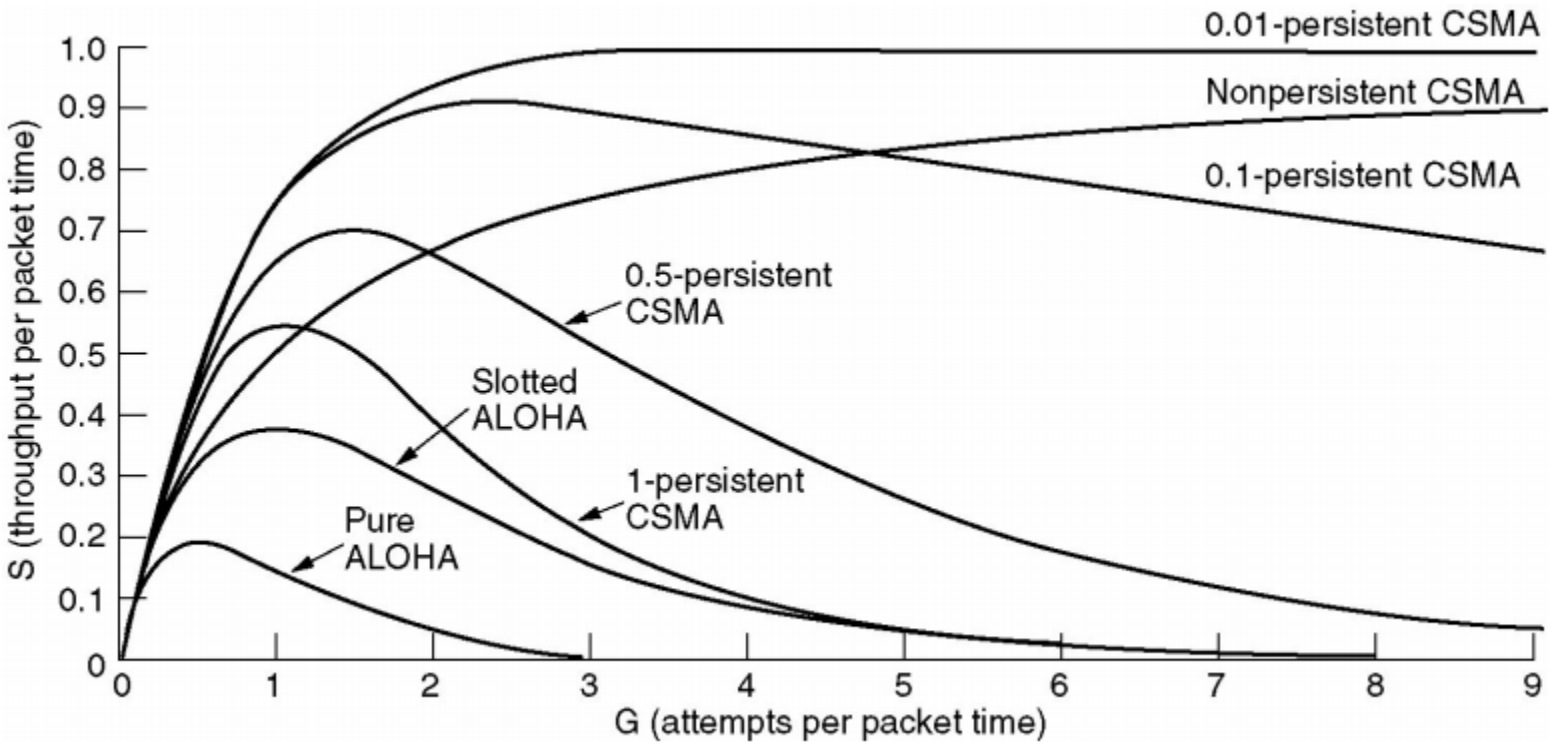
Nem-perzisztens:
Átvitel ha szabad
Különben: késleltetés, újrapróbáljuk



1-perzisztens:
Átvitel amint a csatorna szabad
Ütközés esetén visszalépés,
majd újrapróbáljuk

p-perzisztens:
Átvitel p valószínűséggel, ha a csatorna szabad
Különben: várunk 1 időegységet és újrapróbáljuk

CSMA és ALOHA protokollok összehasonlítása



CSMA/CD - CSMA ütközés detektálással (CD = Collision Detection)

- Ütközés érzékelés esetén meg lehessen szakítani az adást. („Collision Detection”)
  - Minden állomás küldés közben megfigyeli a csatornát,
  - ha ütközést tapasztal, akkor megszakítja az adást, és véletlen ideig várakozik, majd újra elkezdi leadni a keretét.

- Mikor lehet egy állomás biztos abban, hogy megszerezte magának a csatornát?
  - Az ütközés detektálás minimális ideje az az idő, ami egy jelnek a két legtávolabbi állomás közötti átviteléhez szükséges.

CSMA/CD

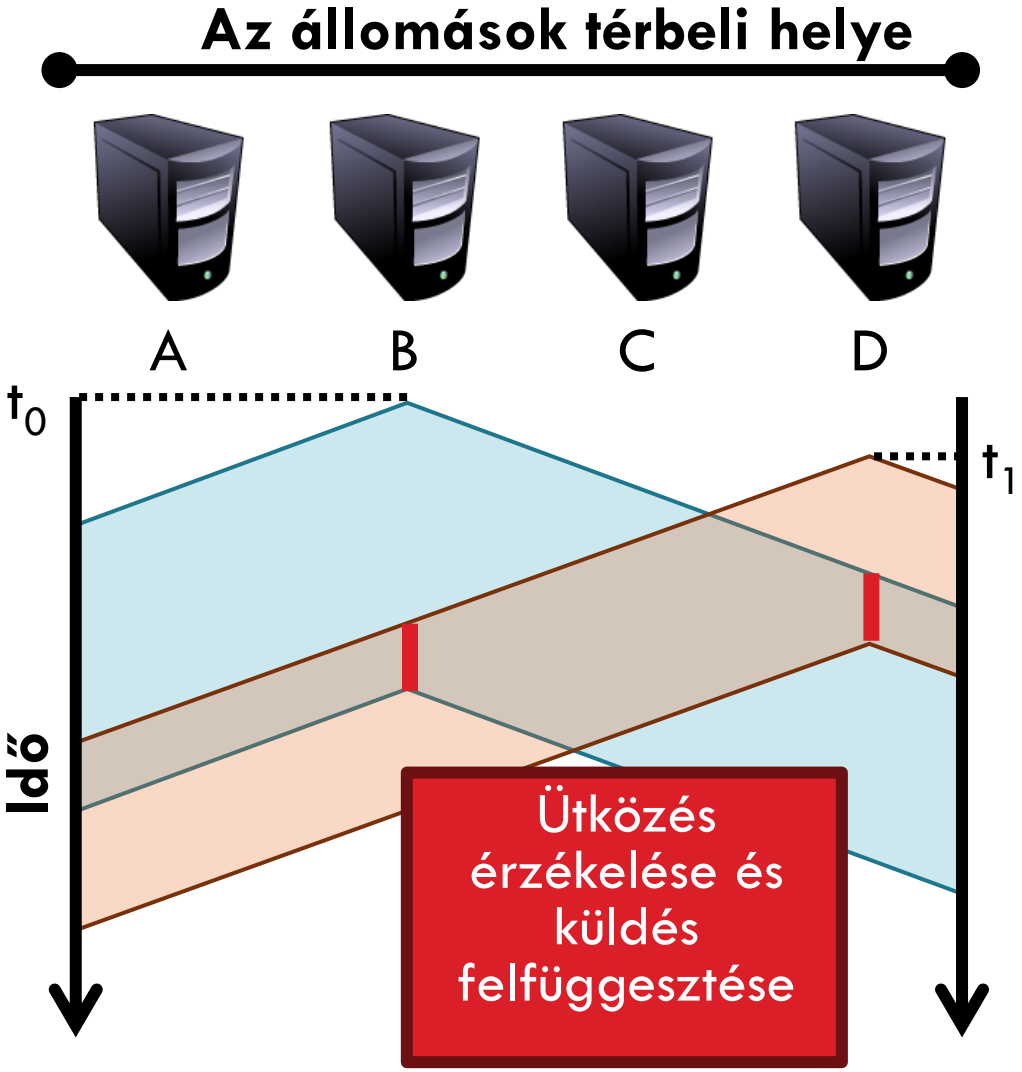
- Egy állomás megszerezte a csatornát, ha minden más állomás érzékeli az átvitelét.
- Az ütközés detektálás működéséhez szükséges a keretek hosszára egy alsó korlátot adnunk
- Ethernet a CSMA/CD-t használja

CSMA/CD

- Carrier sense multiple access with collision detection
- Alapvetés: a közeg lehetőséget ad a csatornába hallgatásra
- Algoritmus
  1. Használjuk valamely CSMA variánst
  2. A keret kiküldése után, figyeljük a közeget, hogy történik-e ütközés
  3. Ha nem volt ütközés, akkor a keretet leszállítottuk
  4. Ha ütközés történt, akkor azonnal megszakítjuk a küldést
     - Miért is folytatnánk hisz a keret már sérült…
  5. Alkalmazzuk az bináris exponenciális hátralék módszert az újraküldés során (binary exponential backoff)

CSMA/CD Ütközések

- Ütközések történhetnek
- Az ütközéseket gyorsan észleljük és felfüggesztjük az átvitelt
- Mi a szerepe a távolságnak, propagációs időnek és a keret méretének?



Binary Exponential Backoff – Bináris exponenciális hátralék

- Ütközés érzékelésekor a küldő egy ún. „jam” jelet küld
  - Minden állomás tudomást szerezzen az ütközésről
- Binary exponential backoff működése:
  - Válasszunk egy $k \in [0, 2^n - 1]$ egyenletes eloszlás szerint, ahol n = az ütközések száma
  - Várjunk k időegységet (keretidőt) az újraküldésig
  - n felső határa 10, 16 sikertelen próbálkozás után pedig eldobjuk a keretet
- A hátralék idő versengési résekre van osztva

Binary Exponential Backoff

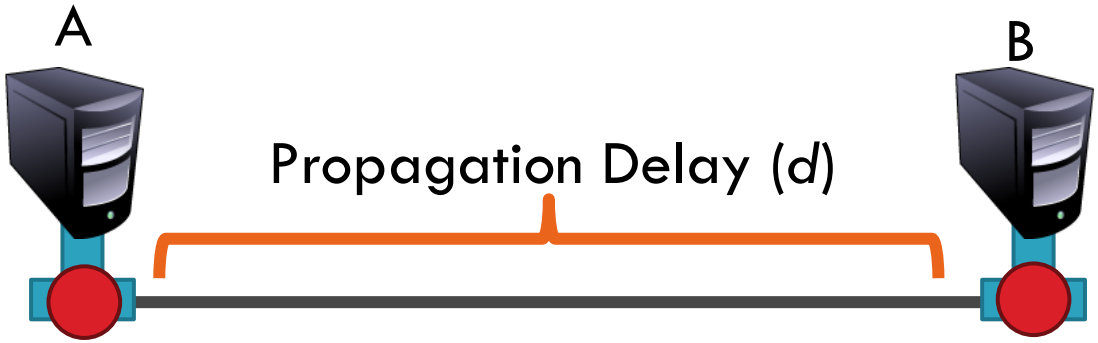
Tekintsünk két állomást, melyek üzenetei ütköztek

- Első ütközés után: válasszunk egyet a két időrés közül
  - A siker esélye az első ütközés után: 50%
  - Átlagos várakozási idő: 1,5 időrés
- Második ütközés után: válasszunk egyet a négy rés közül
  - Sikeres átvitel esélye ekkor: 75%
  - Átlagos várakozási idő: 2,5 rés
- Általában az m. ütközés után:
  - A sikeres átvitel esélye: $1-2^{-m}$
  - Average delay (in slots): $0,5 + 2^{(m-1)}$

Minimális keretméret

- Miért 64 bájt a minimális keretméret?
  - Az állomásoknak elég időre van szüksége az ütközés detektálásához
- Mi a kapcsolat a keretméret és a kábelhossz között?

1. *t időpont*: Az A állomás megkezdi az átvitelt
2. *t + d időpont*: A B állomás is megkezdi az átvitelt
3. *t + 2*d időpont*: A érzékeli az ütközést



Alapötlet: Az A állomásnak 2*d ideig kell küldenie!

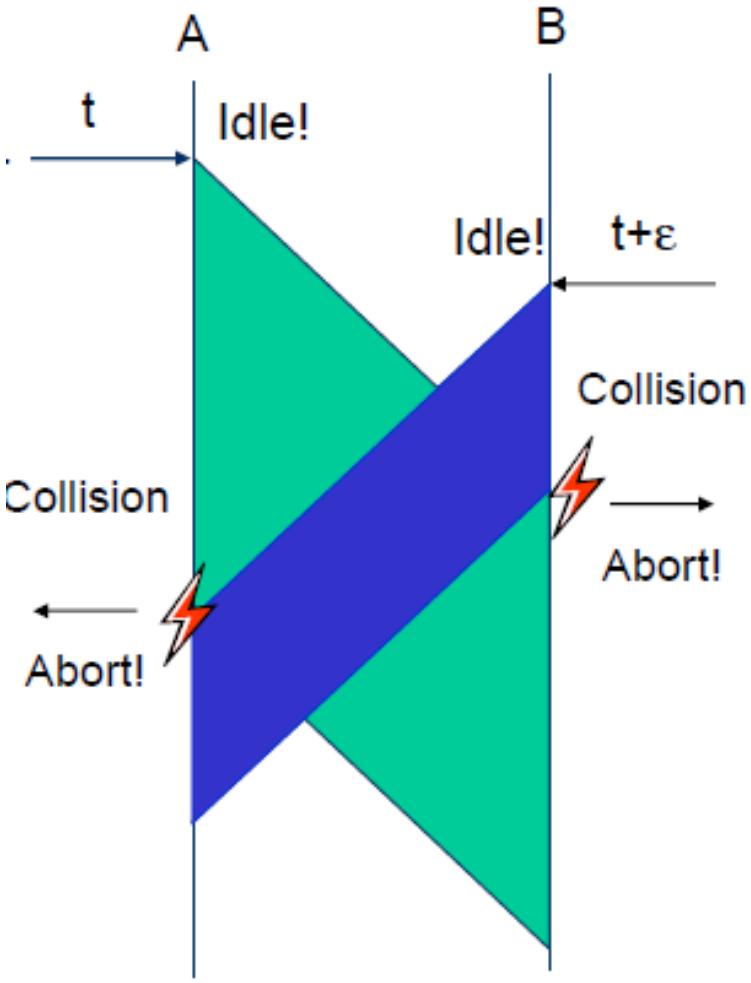
CSMA/CD

- CSMA/CD három állapota: versengés, átvitel és szabad.

- Ahhoz, hogy minden ütközést észleljünk szükséges:

$$T_f \geq 2T_{pg}$$

  - ahol T_f egy keret elküldéséhez szükséges idő
  - és T_{pg} a propagációs késés A és B állomások között



Minimális keretméret

- Az A küldésének 2*d ideig kell tartania
  - Min_keret = ráta (b/s) * 2 * d (s)
    - …
    - Pr…
  - Aza…
  - Min_keret = r… * 2 * távolság (m) / fényseb. (m/s)
- Azaz a kábel össz… a ….
  - Távolság = min_ker… * fénysebesség /(2 * ráta)

- 10 Mbps Ethernet
- A keretméret és a kábelhossz változik a gyorsabb szabványokkal…

$(64B*8)*(2*10^8 mps)/(2*10^7 bps) = 5120$ méter

Minimális keretméret

- Az A küldésének 2*d ideig kell tartania
  - Min_keret = ráta (b/s) * 2 * d (s)
    - … de mi az a d? propagációs késés, melyet a fénysebesség ismeretében ki tudunk számolni
    - Propagációs késés (d) = távolság (m) / fénysebesség (m/s)
  - Azaz:
  - Min_keret = ráta (b/s) * 2 * távolság (m) / fényseb. (m/s)
- Azaz a kábel összhossza ….
  - Távolság = min_keret * fénysebesség /(2 * ráta)

$(64B*8)*(2*10^8 mps)/(2*10^7 bps) = 5120$ méter

Kábelhossz példa

min_keret*fénysebesség/(2*ráta) = max_kábelhossz

(64B*8)*(2*10^8mps)/(2*10Mbps) = 5120 méter

- Mi a maximális kábelhossz, ha a minimális keretméret 1024 bájtra változik?
  - 81,9 kilométer
- Mi a maximális kábelhossz, ha a ráta 1 Gbps-ra változik?
  - 51 méter
- Mi történik, ha mindkettő változik egyszerre?
  - 819 méter

Maximális keretméret

- Maximum Transmission Unit (MTU): 1500 bájt
- Pro:
  - Hosszú csomagokban levő biz hibák jelentős javítási költséget okozhatnak (pl. túl sok adatot kell újraküldeni)
- Kontra:
  - Több bájtot vesztegetünk el a fejlécekben
  - Összességében nagyobb csomag feldolgozási idő
- Adatközpontokban Jumbo keretek
  - 9000 bájtos keretek

Ütközésmentes protokollok

MOTIVÁCIÓ

- az ütközések hátrányosan hatnak a rendszer teljesítményére
  - hosszú kábel, rövid keret
- a CSMA/CD nem mindenhol alkalmazható

FELTÉTELEZÉSEK

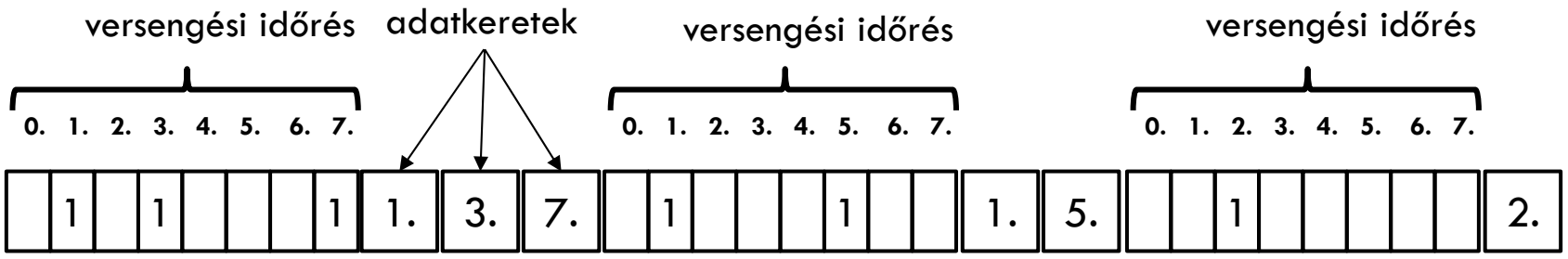
- N állomás van.
- Az állomások 0-ától N-ig egyértelműen sorszámozva vannak.
- Réselt időmodellt feltételezünk.

Alapvető bittérkép protokoll - Egy helyfoglalásos megoldás

- alapvető bittérkép eljárás

MŰKÖDÉS

- Az ütköztetési periódus *N* időrés
- Ha az *i*-edik állomás küldeni szeretne, akkor a *i*-edik versengési időrésben egy *1*-es bit elküldésével jelezheti. (*adatszórás*)
- A versengési időszak végére minden állomás ismeri a küldőket. A küldés a sorszámok szerinti sorrendben történik meg.



Bináris visszaszámlálás protokoll 1/2

- alapvető bittérkép eljárás hátrány, hogy az állomások számának növekedésével a versengési periódus hossza is nő

MŰKÖDÉS

- Minden állomás azonos hosszú bináris azonosítóval rendelkezik.
- A forgalmazni kívánó állomás elkezdi a bináris címét bitenként elküldeni a legnagyobb helyi értékű bittel kezdve. Az azonos pozíciójú bitek logikai *VAGY* kapcsolatba lépnek ütközés esetén. Ha az állomás nullát küld, de egyet hall vissza, akkor feladja a küldési szándékát, mert van nála nagyobb azonosítóval rendelkező küldő.

A HOSZT (0011)	0	–	–	–
B HOSZT (0110)	0	–	–	–
	1	0	1	0
C HOSZT (1010)	1	0	1	1
D HOSZT (1011)	**1**	**0**	**1**	**1**

D kerete

Bináris visszaszámlálás protokoll 2/2

- **Következmény:** a magasabb címmel rendelkező állomásoknak a prioritásuk is magasabb az alacsonyabb című állomásokénál

MOK ÉS WARD MÓDOSÍTÁSA

- Virtuális állomás címek használata.
- Minden sikeres átvitel után ciklikusan permutáljuk az állomások címét.

	A	B	C	D	E	F	G	H
Kezdeti állapot	100	010	111	101	001	000	011	110

Idő

Korlátozott versenyes protokollok

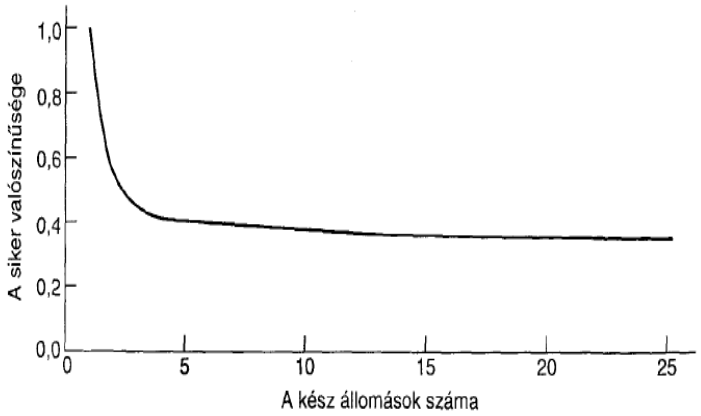
- **Cél:** Ötvözni a versenyhelyzetes és ütközésmentes protokollok jó tulajdonságait.
- **korlátozott versenyes protokoll** – Olyan protokoll, amely kis terhelés esetén versenyhelyzetes technikát használ a kis késleltetés érdekében, illetve nagy terhelés mellett ütközésmentes technikát alkalmaz a csatorna jó kihasználása érdekében.

SZIMMETRIKUS PROTOKOLLOK

- Adott résben k állomás verseng, minden állomás p valószínűséggel adhat. A csatorna megszerzésének valószínűsége: $kp(1-p)^{k-1}$.

$$P(\text{siker optimális } p \text{ mellett}) = \left(\frac{k-1}{k}\right)^{k-1}$$

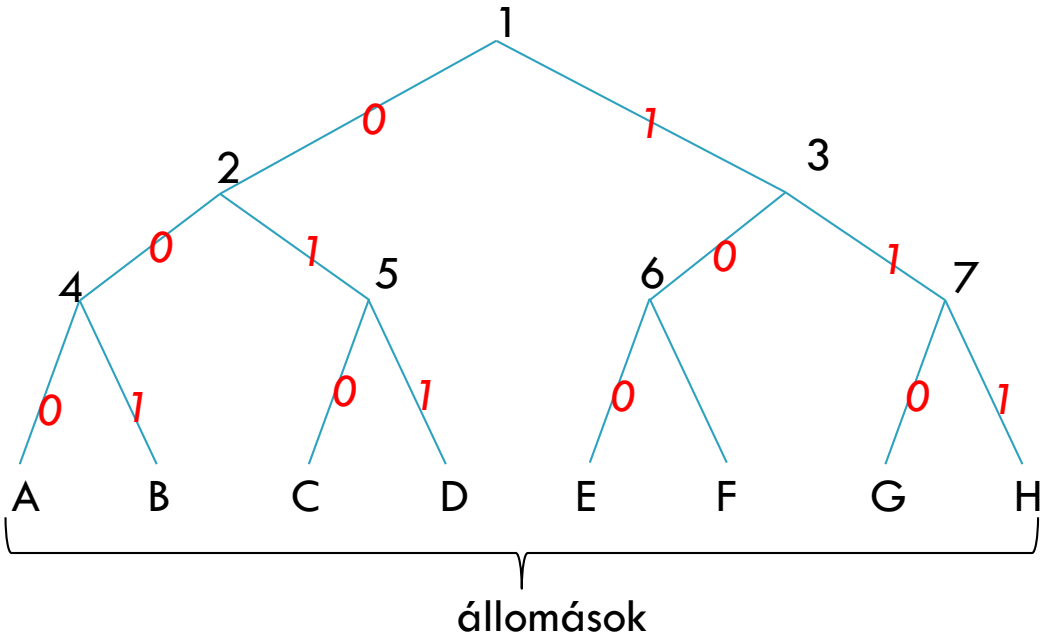
- Azaz a csatorna megszerzésének esélyeit a versenyhelyzetek számának csökkentésével érhetjük el.



Adaptív fabejárási protokoll 1/2

Történeti háttér

- 1943 – Dorfman a katonák szifiliszes fertőzöttségét vizsgálta.
- 1979 – Capetanakis bináris fa reprezentáció az algoritmus számítógépes változatával.



Adaptív fabejárási protokoll 2/2

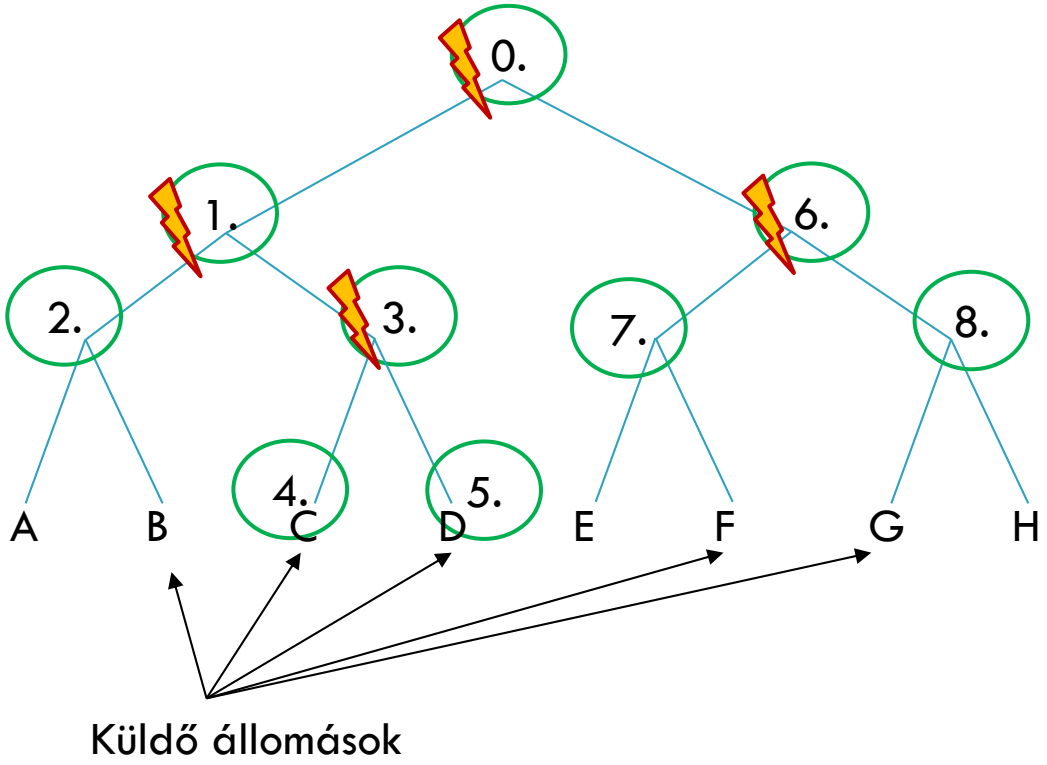
Működés

- *0*-adik időrésben mindenki küldhet.
  - Ha ütközés történik, akkor megkezdődik a fa *mélységi bejárása*.
- A rések a fa egyes csomópontjaihoz vannak rendelve.
- Ütközéskor rekurzívan az adott csomópont bal illetve jobb gyerekcsomópontjánál folytatódik a keresés.
- Ha egy bitrés kihasználatlan marad, vagy pontosan egy állomás küld, akkor a szóban forgó csomópont keresése befejeződik.

Következmény

- Minél nagyobb a terhelés, annál mélyebben érdemes kezdeni a keresést.

Adaptív fabejárás példa



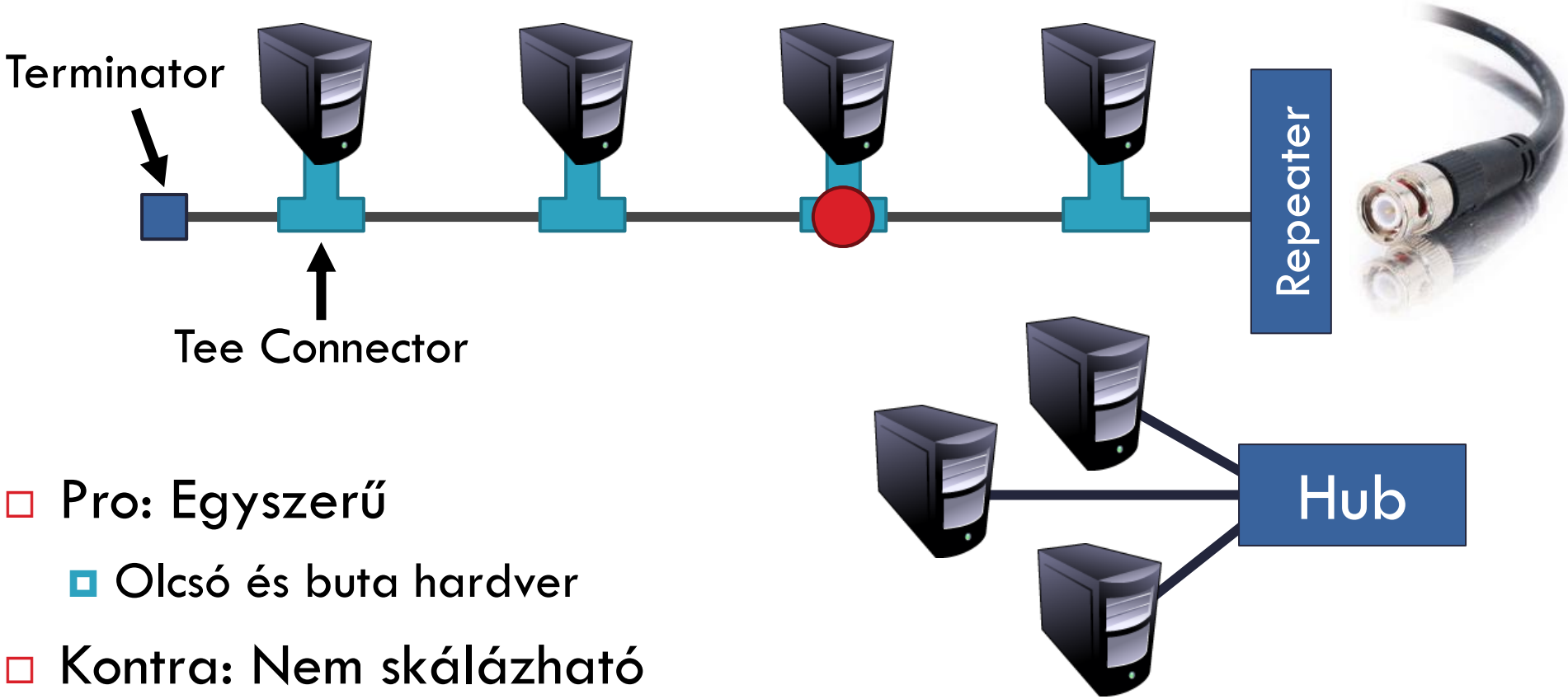
Az adatkapcsolati réteg „legtetején”…



- Bridging, avagy hidak
  - Hogyan kapcsoljunk össze LAN-okat?
- Funkciók:
  - Keretek forgalomirányítása a LAN-ok között
- Kihívások:
  - Plug-and-play, önmagát konfiguráló
  - Esetleges hurkok feloldása

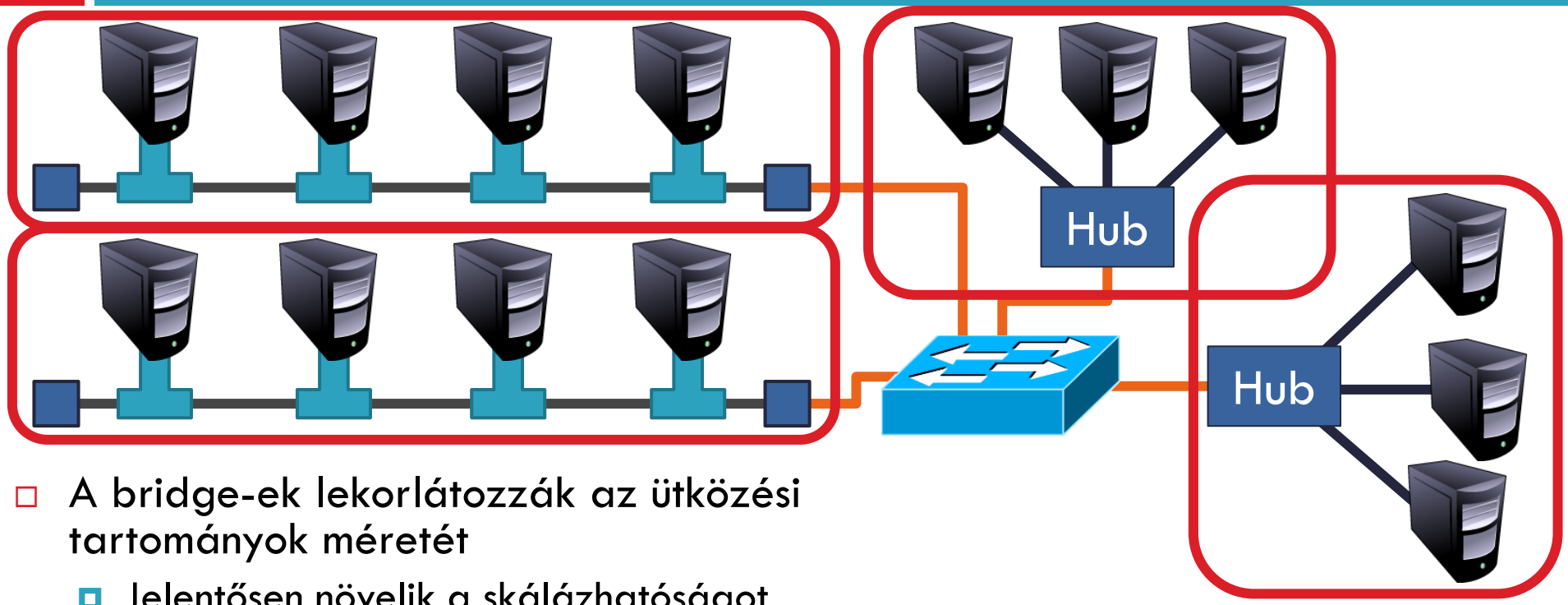
Visszatekintés

- Az Ethernet eredetileg adatszóró technológia volt



- Pro: Egyszerű
  - Olcsó és buta hardver
- Kontra: Nem skálázható
  - Több állomás = több ütközés = káosz

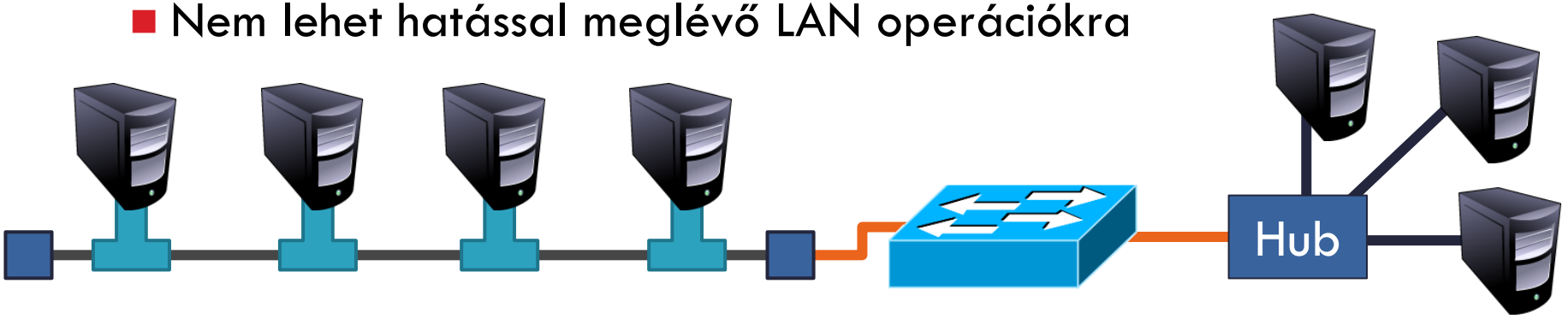
LAN-ok összekapcsolása



- A bridge-ek lekorlátozzák az ütközési tartományok méretét
  - Jelentősen növelik a skálázhatóságot
  - Kérdés: lehetne-e az egész Internet egy bridge-ekkel összekötött tartomány?
- Hátrány: a bridge-ek sokkal komplexebb eszközök a hub-oknál
  - Fizikai réteg VS Adatkapcsolati réteg
  - Memória pufferek, csomag feldolgozó hardver és routing (útválasztó) táblák szükségesek

Bridge-ek (magyarul: hidak)

- Az Ethernet switch eredeti formája
- Több IEEE 802 LAN-t kapcsol össze a 2. rétegben
- Célok
  - Ütközési tartományok számának csökkentése
  - Teljes átlátszóság
    - "Plug-and-play," önmagát konfiguráló
    - Nem szükségesek hw és sw változtatások a hosztokon/hub-okon
    - Nem lehet hatással meglévő LAN operációkra

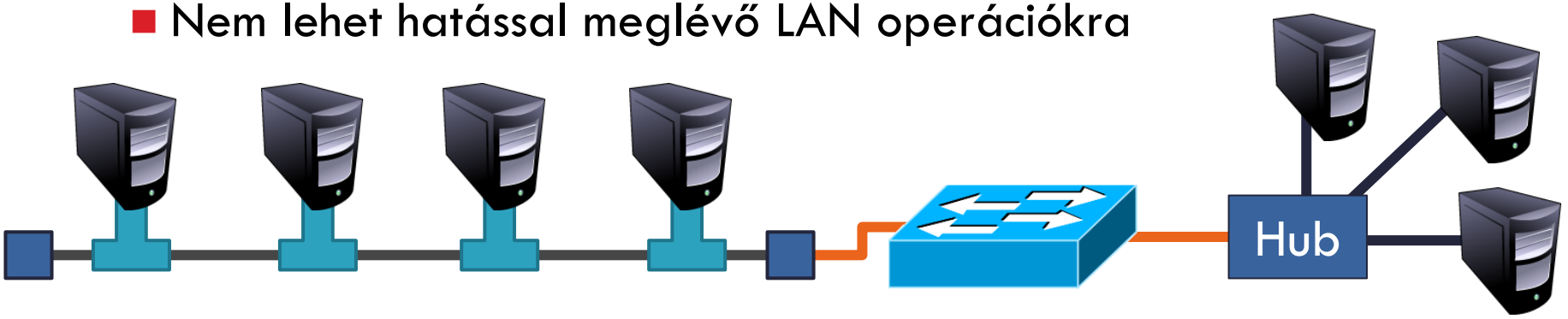


Bridge-ek (magyarul: hidak)

- Az Ethernet switch eredeti formája
- [illegible]
- [illegible]

1. Keretek továbbítása
2. (MAC) címek tanulása
3. Feszítőfa (Spanning Tree) Algoritmus (a hurkok kezelésére)

  - Nem szükségesek hw és sw változtatások a hosztokon/hub-okon
  - Nem lehet hatással meglévő LAN operációkra



Keret Továbbító Táblák

- Minden bridge karbantart egy továbbító táblát (forwarding table)

MAC Cím	Port	Kor
00:00:00:00:00:AA	1	1 perc
00:00:00:00:00:BB	2	7 perc
00:00:00:00:00:CC	3	2 mp
00:00:00:00:00:DD	1	3 perc

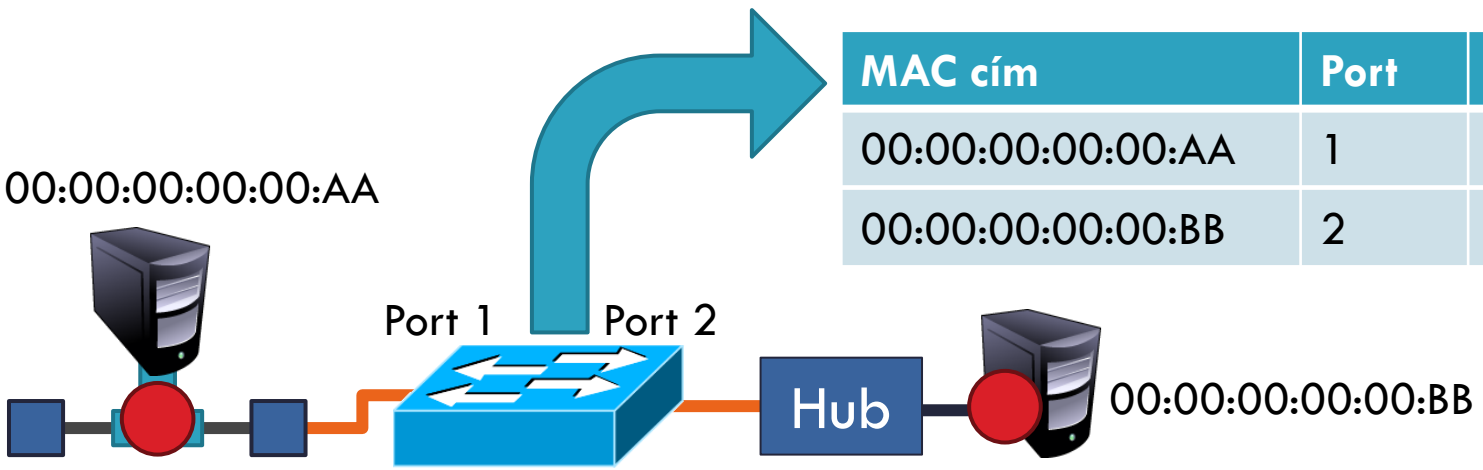


Címek tanulása

- Kézi beállítás is lehetséges, de…
  - Időigényes
  - Potenciális hiba forrás
  - Nem alkalmazkodik a változásokhoz (új hosztok léphetnek be és régiek hagyhatják el a hálózatot)
- Ehelyett: tanuljuk meg a címeket
  - Tekintsük a forrás címeit a különböző portok
    kereteknek --- képezzünk ebből egy táblázat

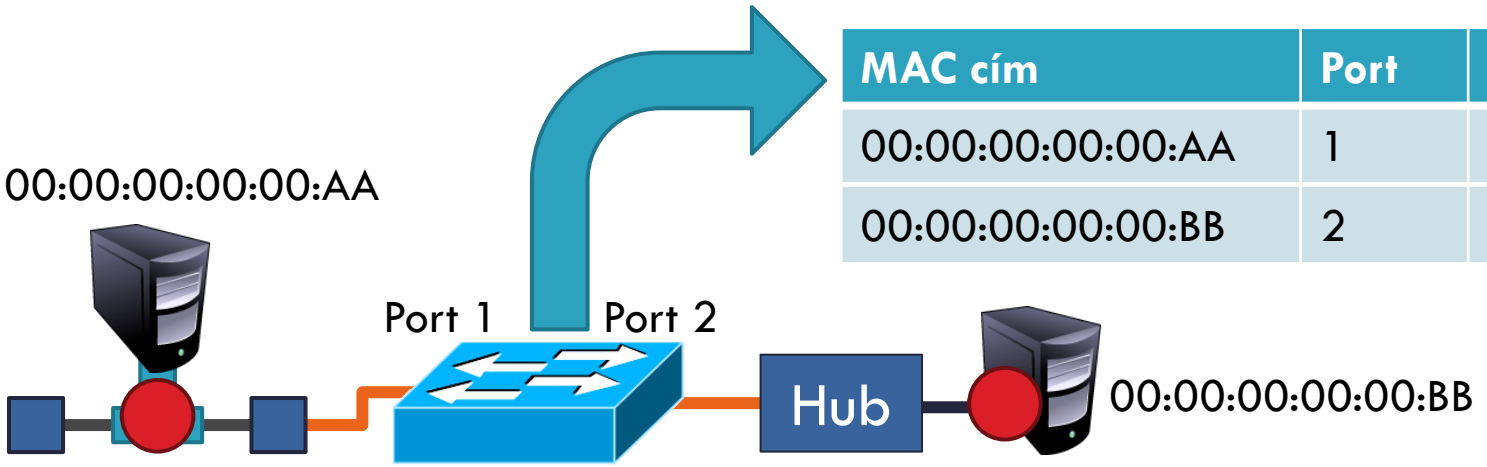
Töröljük a régi bejegyzéseket

MAC cím	Port	Kor
00:00:00:00:00:AA	1	0 minutes
00:00:00:00:00:BB	2	0 minutes



Címek tanulása

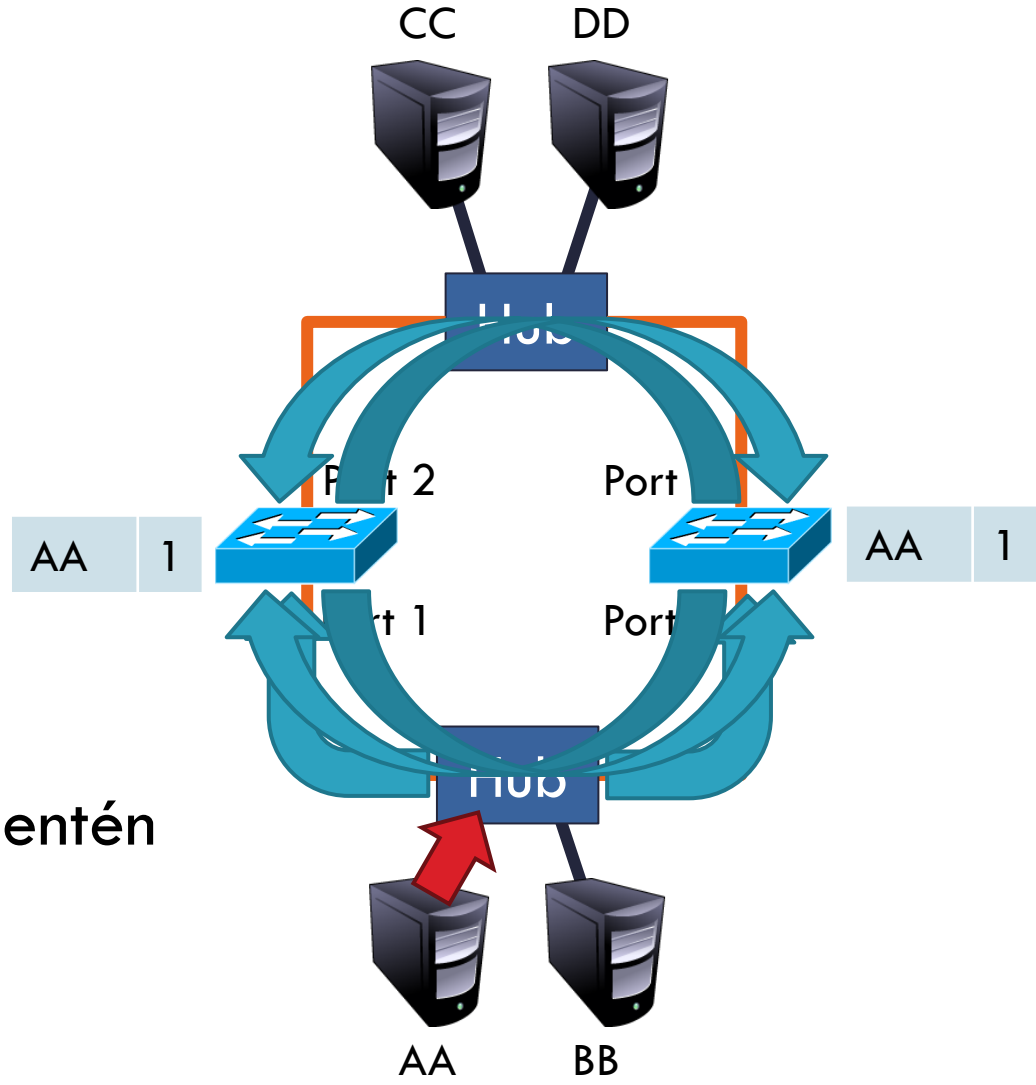
- Kézi beállítás is lehetséges, de…
  - Időigényes
  - Potenciális hiba forrás
  - Nem alkalmazkodik a változásokhoz (új hosztok léphetnek be és régiek hagyhatják el a hálózatot)
- Ehelyett: tanuljuk meg a címeket
  - Tekintsük a forrás címeit a különböző portokon beérkező kereteknek --- képezzünk ebből egy táblázatot



MAC cím	Port	Kor
00:00:00:00:00:AA	1	0 minutes
00:00:00:00:00:BB	2	0 minutes

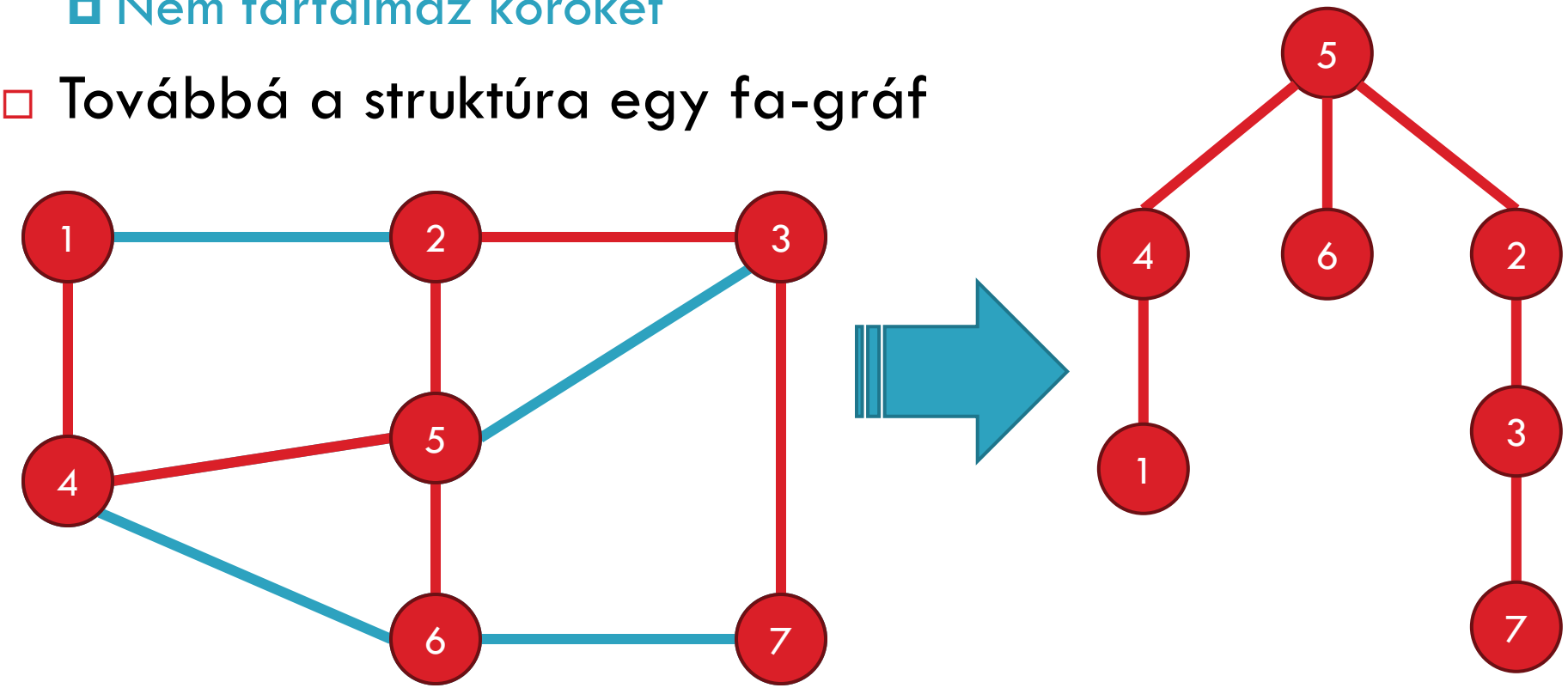
Hurkok problémája

- <Src=AA, Dest=DD>
- Ez megy a végtelenségig
  - Hogyan állítható meg?
- Távolítsuk el a hurkokat a topológiából
  - A kábelek kihúzása nélkül
- 802.1 (LAN) definiál egy algoritmust feszítőfa építéséhez és karbantartásához, mely mentén lehetséges a keretek továbbítása



Feszítőfa

- Egy gráf éleinek részhalmaza, melyre teljesül:
  - Lefed minden csomópontot
  - Nem tartalmaz köröket
- Továbbá a struktúra egy fa-gráf



A 802.1 feszítőfa algoritmusa

1. Az egyik bride-et megválasztjuk a fa gyökerének
2. Minden bridge megkeresi a legrövidebb utat a gyökérhez
3. Ezen utak unióját véve megkapjuk a feszítőfát

- A fa építése során a bridge-ek egymás között konfigurációs üzeneteket (Configuration Bridge Protocol Data Units [BPDUs]) cserélnek
  - A gyökér elem megválasztásához
  - A legrövidebb utak meghatározásához
  - A gyökérhez legközelebbi szomszéd (next hop) állomás és a hozzá tartozó port azonosításához
  - A feszítőfához tartozó portok kiválasztása

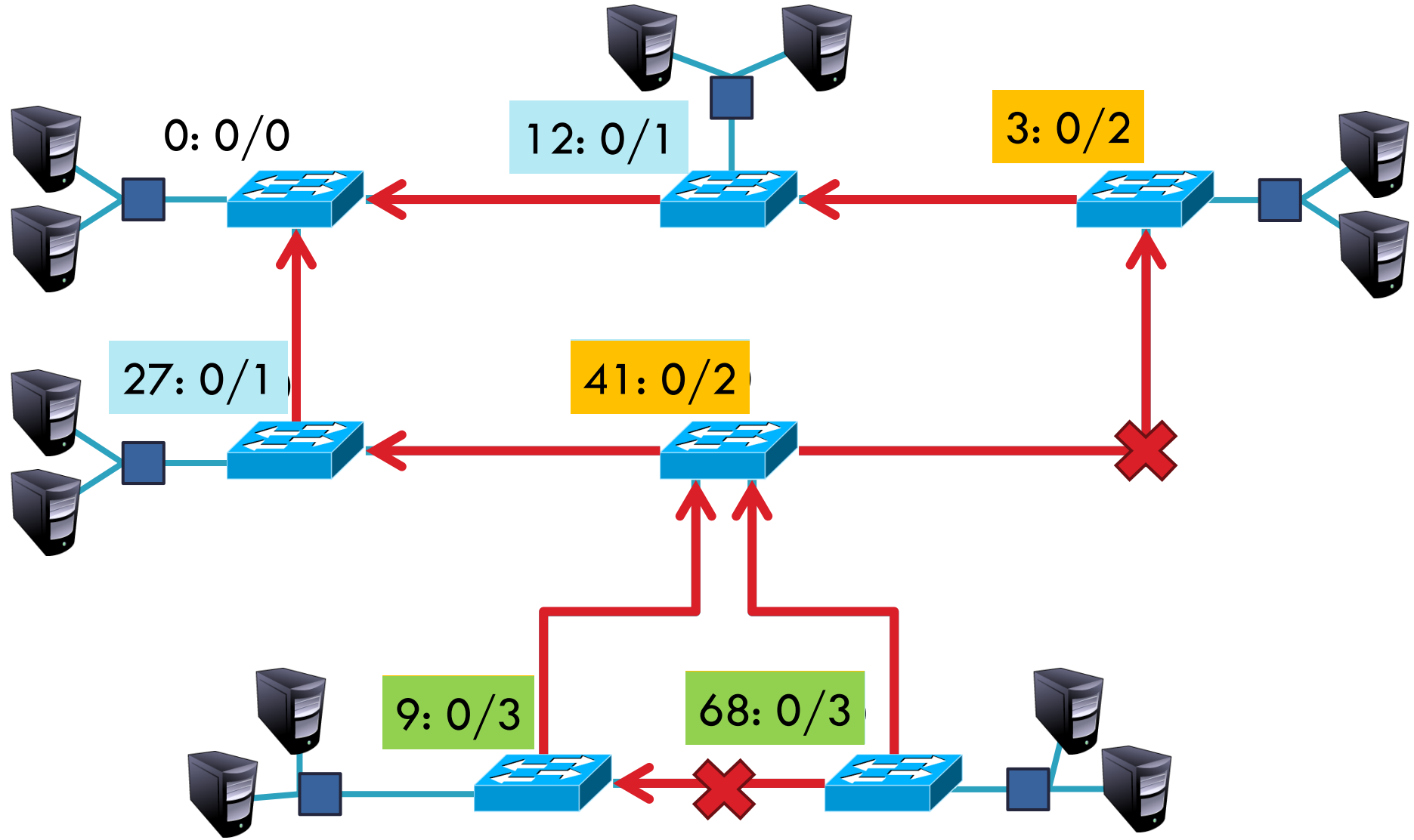
Gyökér meghatározása

- Kezdetben minden állomás feltételezi magáról, hogy gyökér
- Bridge-ek minden irányba szétküldik a BPDU üzeneteiket:



- A fogadott BPDU üzenet alapján, minden switch választ:
  - Egy új gyökér elemet (legkisebb ismert Gyökér ID alapján)
  - Egy új gyökér portot (melyik interfész megy a gyökér irányába)
  - Egy új kijelölt bridge-et (a következő állomás a gyökérhez vezető úton)

Feszítőfa építése



Bridge-ek vs. Switch-ek
Hidak vs. Kapcsolók

- A bridge-ek lehetővé teszik hogy növeljük a LAN-ok kapacitását
  - Csökkentik a sikeres átvitelhez szükséges elküldendő csomagok számát
  - Kezeli a hurkokat
- A switch-ek a bridge-ek speciális esetei
  - Minden port egyetlen egy hoszthoz kapcsolódik
    - Lehet egy kliens terminál
    - vagy akár egy másik switch
  - Full-duplex link-ek
  - Egyszerűsített hardver: nincs szükség CSMA/CD-re!
  - Különböző sebességű/rátájú portok is lehetségesek

Kapcsoljuk össze az Internetet

- Switch-ek képességei:
  - MAC cím alapú útvonalválasztás a hálózatban
  - Automatikusan megtanulja az utakat egy új állomáshoz
  - Feloldja a hurkokat
- Lehetne a teljes internet egy ily módon összekötött tartomány?

NEM

Korlátok

- Nem hatékony
  - Elárasztás ismeretlen állomások megtalálásához
- Gyenge teljesítmény
  - A feszítőfa nem foglalkozik a terhelés elosztással
  - Hot spots
- Nagyon gyenge skálázhatóság
  - Minden switch-nek az Internet összes MAC címét ismerni kellene a továbbító táblájában!
- Az IP fogja ezt a problémát megoldani…

Köszönöm a figyelmet!

Hálózati réteg



- Szolgáltatás
  - Csomagtovábbítás
  - Útvonalválasztás
  - Csomag fragmentálás kezelése
  - Csomag ütemezés
  - Puffer kezelés
- Interfész
  - Csomag küldése egy adott végpontnak
- Protokoll
  - Globálisan egyedi címeket definiálása
  - Routing táblák karbantartása
- Példák: Internet Protocol (IPv4), IPv6

Forgalomirányító algoritmusok

DEFINÍCIÓ

A hálózati réteg szoftverének azon része, amely azért a döntésért felelős, hogy a bejövő csomag melyik kimeneti vonalon kerüljön továbbításra.

- A folyamat két jól-elkülöníthető lépésre bontható fel:
  1. Forgalomirányító táblázatok feltöltése és karbantartása.
  2. Továbbítás.

ELVÁRÁSOK

helyesség, egyszerűség, robosztusság, stabilitás, igazságosság, optimalitás és hatékonyság

ALGORITMUS OSZTÁLYOK

1. Adaptív algoritmusok
   - A topológia és rendszerint a forgalom is befolyásolhatja a döntést
2. Nem-adaptív algoritmusok
   - offline meghatározás, betöltés a router-ekbe induláskor

Forgalomirányító algoritmusok

KÜLÖNBSÉGEK AZ EGYES ADAPTÍV ALGORITMUSOKBAN

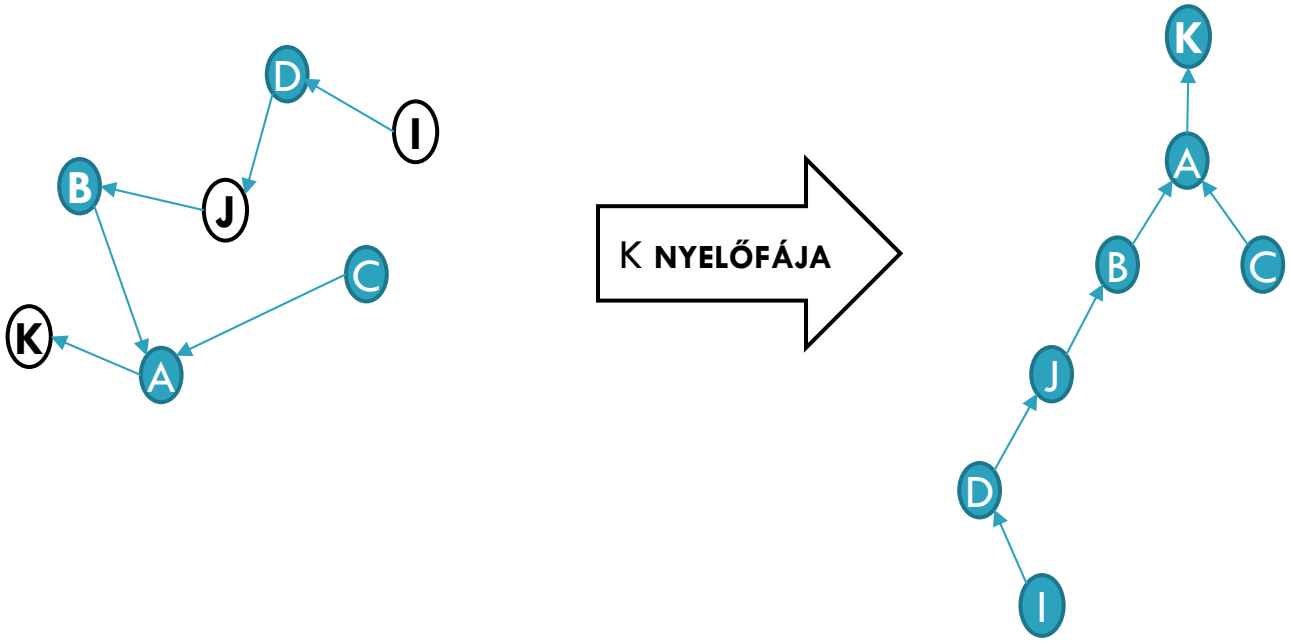
1. Honnan kapják az információt?
   - szomszédok, helyileg, minden router-től
2. Mikor változtatják az útvonalakat?
   - meghatározott másodpercenként, terhelés változásra, topológia változásra
3. Milyen mértékeket használnak az optimalizáláshoz?
   - távolság, ugrások (*hops*) száma, becsült késleltetés

Optimalitási elv

Ha ***J*** router az ***I*** router-től ***K*** router felé vezető *optimális útvonalon* helyezkedik el, akkor a J-től a K-ig vezető útvonal ugyanerre esik.

- **Következmény**

  Az összes forrásból egy célba tartó optimális utak egy olyan fát alkotnak, melynek a gyökere a cél. Ezt nevezzük ***nyelőfá***nak.

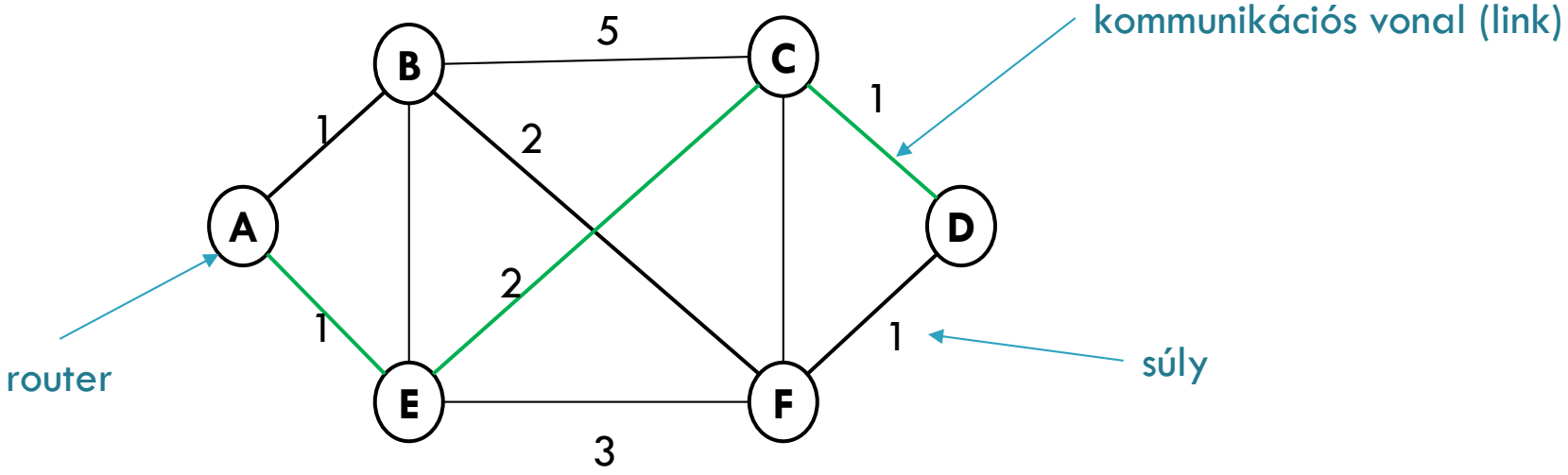


Legrövidebb út alapú forgalomirányítás

ALHÁLÓZAT REPREZENTÁCIÓJA

Az alhálózat tekinthető egy gráfnak, amelyben minden router egy csomópontnak és minden él egy kommunikációs vonalnak (link) felel meg. Az éleken értelmezünk egy $w: E \rightarrow \mathbb{R}_0^+$ nem-negatív súlyfüggvényt, amelyek a legrövidebb utak meghatározásánál használunk.

- *G=(V,E)* gráf reprezentálja az alhálózatot
- *P* útvonal súlya: $w(P) = \sum_{e \epsilon P} w(e)$



Távolságvektor alapú forgalomirányítás

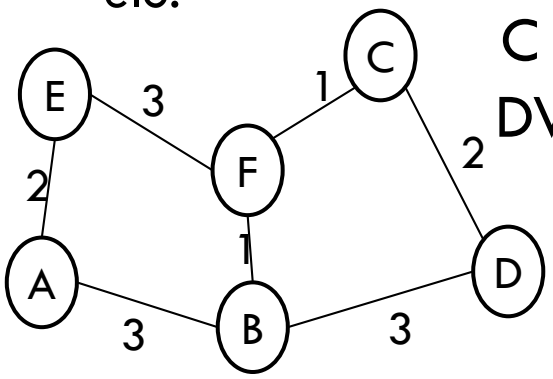
- Dinamikus algoritmusoknak 2 csoportja van:
  - távolságvektor alapú illetve (distance vector routing)
  - kapcsolatállapot alapú (link-state routing)

- **Távolságvektor alapú**: Minden router-nek egy táblázatot kell karbantartania, amelyben minden célhoz szerepel a legrövidebb ismert távolság, és annak a vonalnak az azonosítója, amelyiken a célhoz lehet eljutni. A táblázatokat a szomszédoktól származó információk alapján frissítik.
  - Elosztott Bellman-Ford forgalomirányítási algoritmusként is nevezik.
  - ARPANET eredeti forgalomirányító algoritmusa ez volt. RIP (Routing Information Protocol) néven is ezt használták.

Távolságvektor alapú forgalomirányítás Elosztott Bellman-Ford algoritmus

KÖRNYEZET ÉS MŰKÖDÉS

- Minden csomópont csak a közvetlen szomszédjaival kommunikálhat.
- Aszinkron működés.
- Minden állomásnak van saját távolság vektora. Ezt periodikusan elküldi a direkt szomszédoknak.
- A kapott távolság vektorok alapján minden csomópont új táblázatot állít elő.

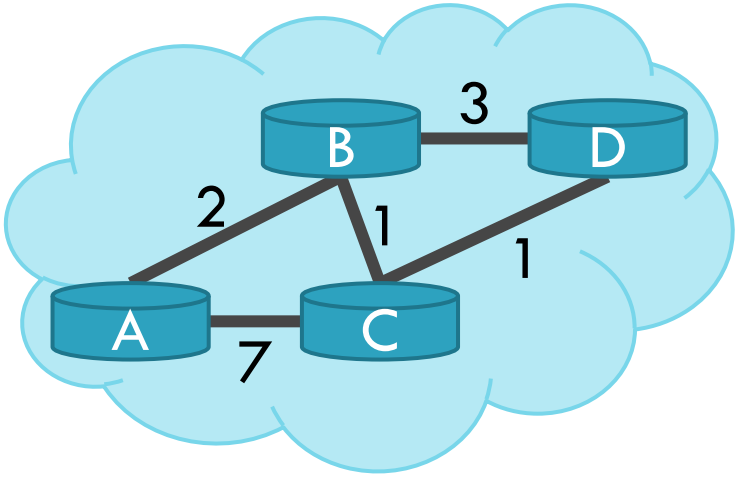


C állomás DV táblája

Cél	Ktsg.
A	5
B	2
D	2
E	4
F	1

- Nincs bejegyzés C-hez
- Kezdetben csak a közvetlen szomszédokhoz van info
  - Más célállomások költsége = ∞
- Végül kitöltött vektort kapunk

Distance Vector Initialization




```
1.   Initialization:
2.     for all neighbors V  do
3.       if V adjacent to A
4.         D(A, V) = c(A,V);
5.     else
6.         D(A, V) = ∞;
…
```

Node A

Dest.	Cost	Next
B	2	B
C	7	C
D	∞	

Node B

Dest.	Cost	Next
A	2	A
C	1	C
D	3	D

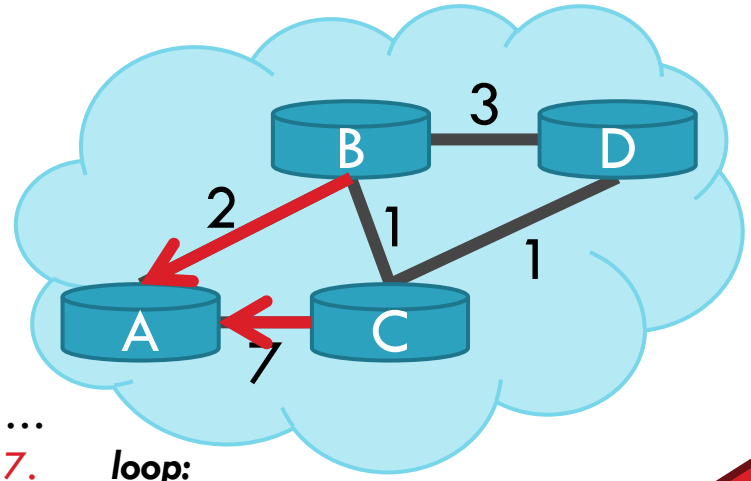
Node C

Dest.	Cost	Next
A	7	A
B	1	B
D	1	D

Node D

Dest.	Cost	Next
A	∞	
B	3	B
C	1	C

Distance Vector: 1st Iteration



Node A

Dest.	Cost	Next
B	2	B
C	3	B
D	5	B

Node B

Dest.	Cost	Next
A	2	A
C	1	C
D	2	C

```
…
7.   loop:
…
12.  else if (update D(V, Y) recei
13.    for all desti
14.      if (destino
15.        D(A,Y) =
16.      else
17.        D(A, Y) =
                   mi
                   D(A, V) + D(V, Y));
18.  if (there is a new min. for dest. Y)
19.    send D(A, Y) to all neighbors
20.  forever
```

D(A,C) = min(D(A,C), D(A,B)+D(B,C))

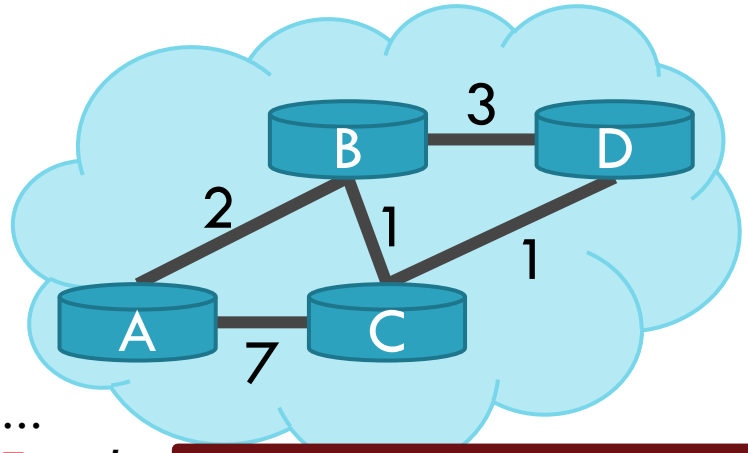
D(A,D) = min(D(A,D), D(A,B)+D(B,D))
= min(8, 3 + 3) = 5

Node D

Dest.	Cost	Next
B	1	B
D	1	D

Dest.	Cost	Next
B	3	B
C	1	C

Distance Vector: End of 3rd Iteration



Node A

Dest.	Cost	Next
B	2	B
C	3	B
D	4	B

Node B

Dest.	Cost	Next
A	2	A
C	1	C
D	2	C

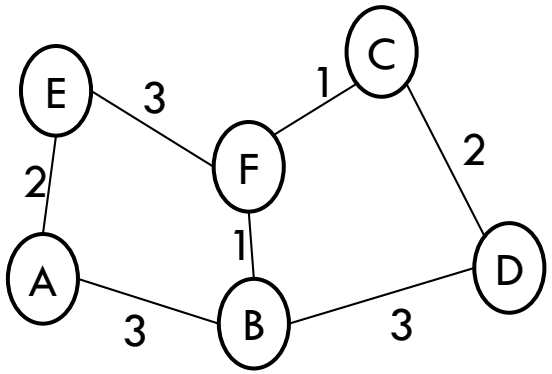
Dest.	Cost	Next
A	3	B
B	1	B
D	1	D

Dest.	Cost	Next
A	4	C
B	2	C
C	1	C

- Nothing changes, algorithm terminates
- Until something changes…

```
…
7.   loop
…
12.  else
13.    fo
14.      i
15.
16.    else
17.      D(A, Y) =
                 min(D(A, Y),
                 D(A, V) + D(V, Y));
18.  if (there is a new min. for dest. Y)
19.    send D(A, Y) to all neighbors
20.  forever
```

Elosztott Bellman-Ford algoritmus – *példa*



Becsült késleltetés A-tól kezdetben

A	cost	N. Hop
B	3	B
C	∞	-
D	∞	-
E	2	E
F	∞	-

B vektora A-nak

A	3
B	0
C	∞
D	3
E	∞
F	1

E vektora A-nak

A	2
B	∞
C	∞
D	∞
E	0
F	3

Új becsült késleltetés A-tól

A	cost	N. Hop
B	3	B
C	∞	-
D	6	B
E	2	E
F	4	B

A vektora B-nek és E-nek

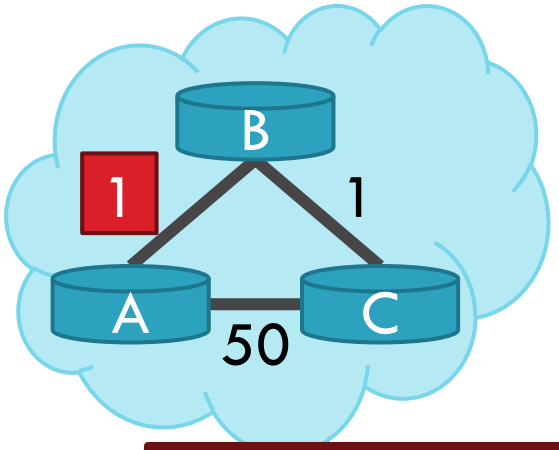
A	0
B	3
C	∞
D	6
E	2
F	4

…

Új becsült késleltetés A-tól

A	cost	N. Hop
B	3	B
C	5	B
D	6	B
E	2	E
F	4	B

```
7.    loop:
8.      wait (link cost update or update message)
9.      if (c(A,V) changes by d)
10.       for all destinations Y through V do
11.         D(A,Y) =  D(A,Y) + d
12.     else if (update D(V, Y) received from V)
13.       for all destinations Y do
14.         if (destination Y through V)
15.           D(A,Y) = D(A,V) + D(V, Y);
16.         else
17.           D(A, Y) = min(D(A, Y), D(A, V) + D(V, Y));
18.
19.
20.
```



Link Co
Algor

Good news travels fast

Algorithm
erminates

Node B

D	C	N
A	4	A
C	1	B

D	C	N
A	1	A
C	1	B

D	C	N
A	1	A
C	1	B

D	C	N
A	1	A
C	1	B

Node C

D	C	N
A	5	B
B	1	B

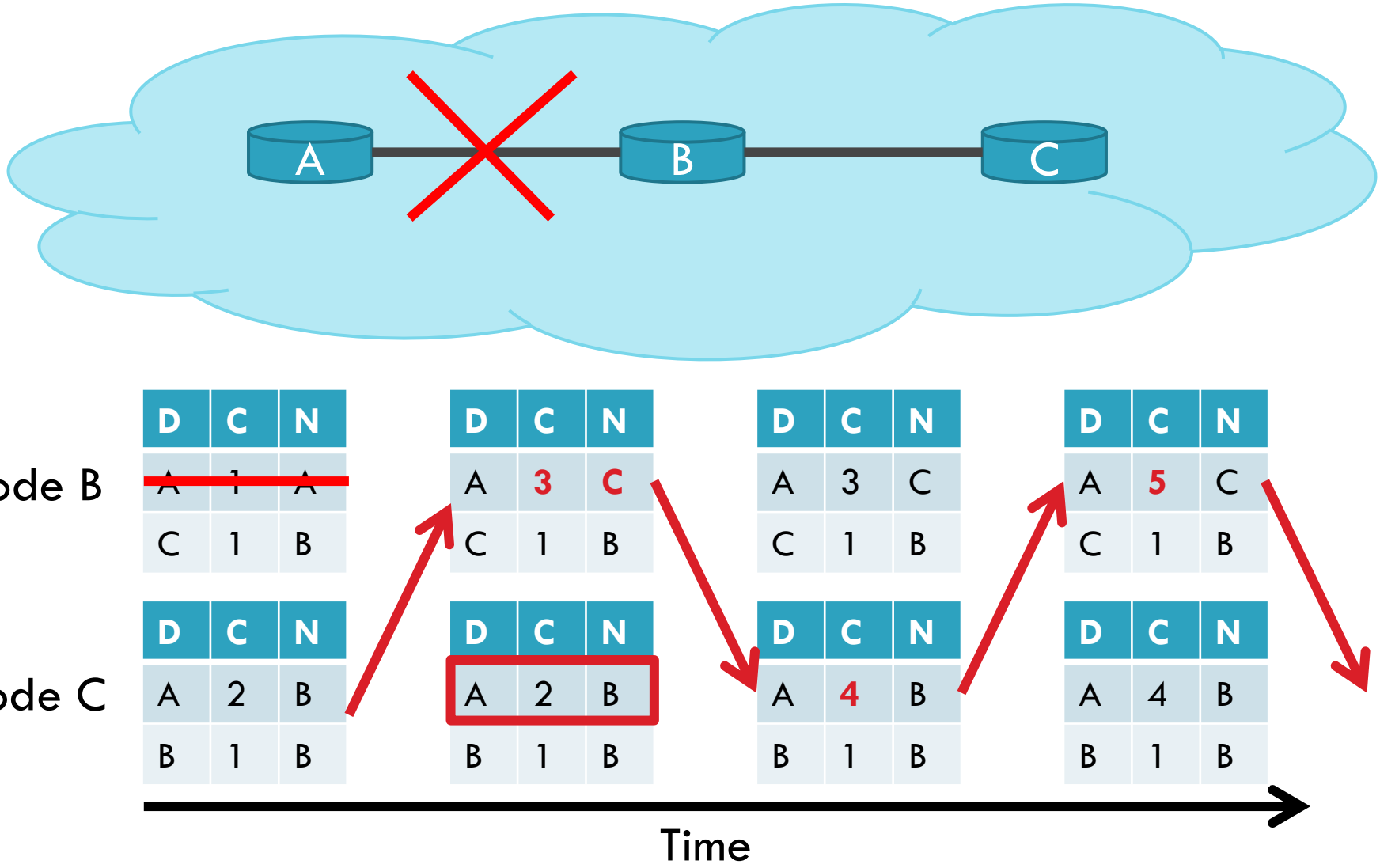
D	C	N
A	5	B
B	1	B

D	C	N
A	2	B
B	1	B

D	C	N
A	2	B
B	1	B

Time

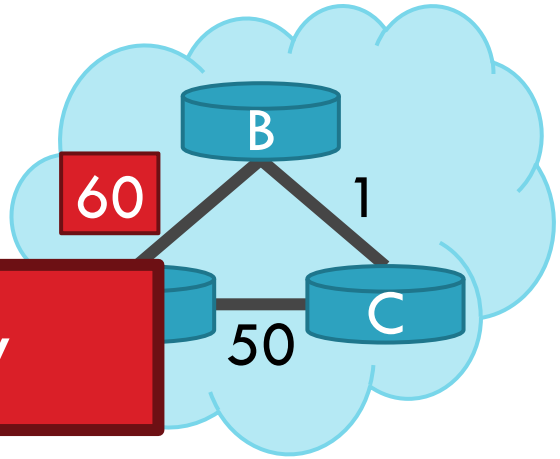
Távolság vektor protokoll – *Végtelenig számolás problémája (count to infinity)*



Példa - Count to Infinity Problem

- Node B knows D(C, A) = 5
- However, B does not know the path is C → B → A
- Thus, D(B,A

Bad news travels slowly



Node B

D	C	N
A	4	A
C	1	B

D	C	N
A	6	C
C	1	B

D	C	N
A	6	C
C	1	B

D	C	N
A	8	C
C	1	B

Node C

D	C	N
A	5	B
B	1	B

D	C	N
A	5	B
B	1	B

D	C	N
A	7	B
B	1	B

D	C	N
A	7	B
B	1	B

Time

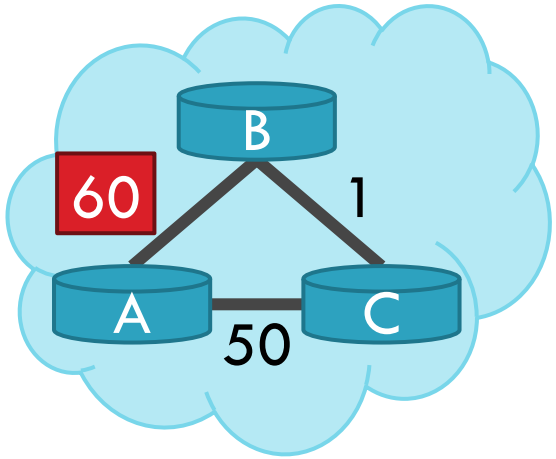
Elosztott Bellman-Ford algoritmus – *Végtelenig számolás problémája*

PROBLÉMA

- A „jó hír” gyorsan terjed.
- A „rossz hír” lassan terjed.
- Azaz ciklusok keletkezhetnek.
- Lehetséges megoldás:
  - **„split horizon with poisoned reverse”**: negatív információt küld vissza arról a szomszédjának, amit tőle „tanult”. (*RFC 1058*)

Split horizon with Poisoned Reverse

- Ha C B-n keresztül irányítja a forgalmat A állomáshoz
  - C állomás B-nek D(C, A) = ∞ távolságot küld
  - Azaz B állomás nem fog C-n keresztül irányítani az A-ba menő forgalmat



Node B

D	C	N
A	4	A
C	1	B

D	C	N
A	60	A
C	1	B

D	C	N
A	60	A
C	1	B

D	C	N
A	51	C
C	1	B

Node C

D	C	N
A	5	B
B	1	B

D	C	N
A	5	B
B	1	B

D	C	N
A	50	A
B	1	B

D	C	N
A	50	A
B	1	B

Time